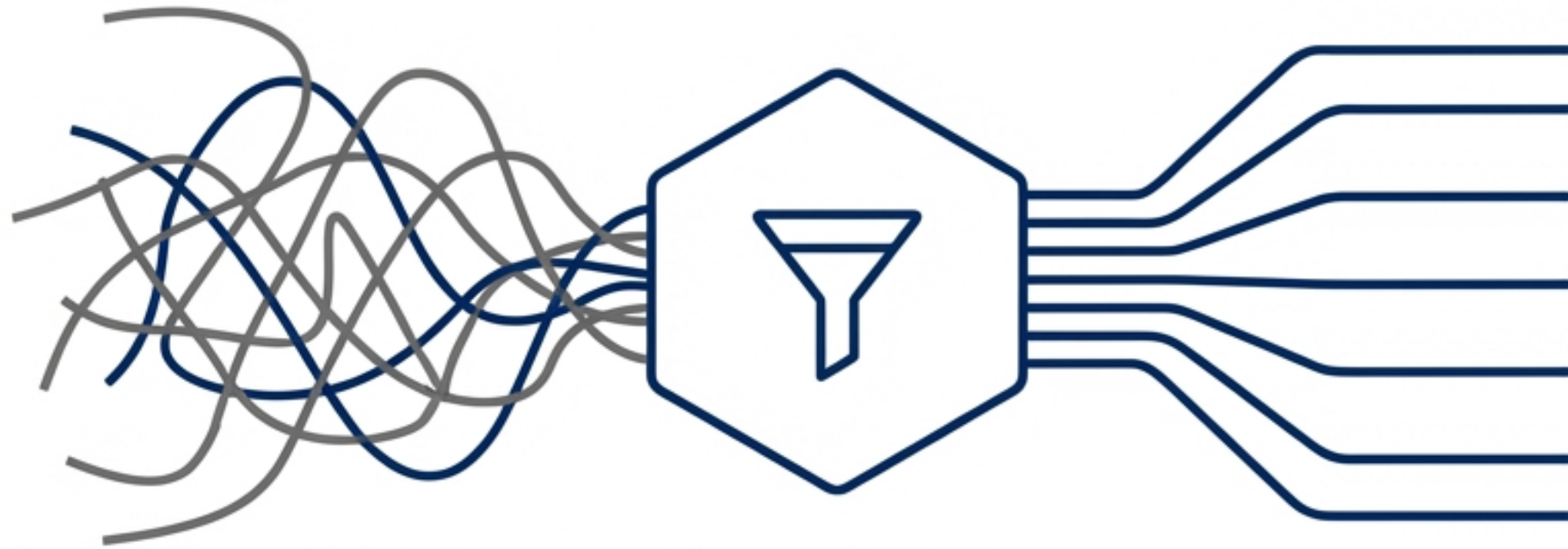




MitmProxy Multi-User Authentication System

Implementation Brief v1.0.0 | Custom Addon Architecture & Deployment

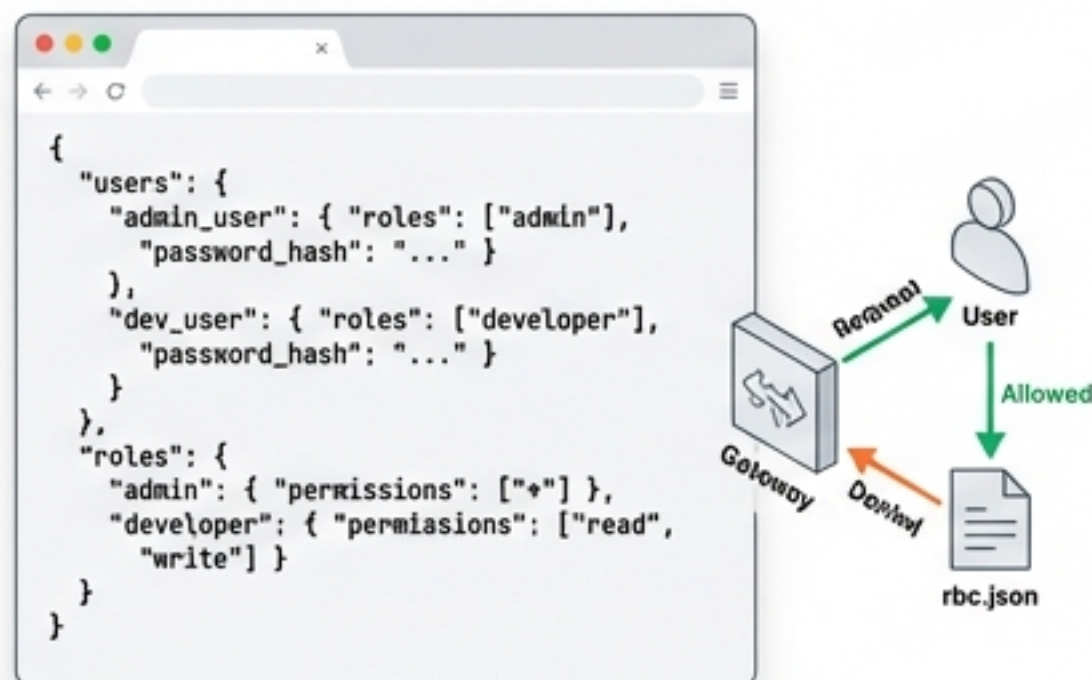
From Vulnerability to Accountability

This project delivers a custom authentication addon for MitmProxy, transforming it from an open relay into a secure, multi-tenant gateway. We achieve **compliance** and security **isolation** while maintaining the **zero-latency performance of the native event loop**.



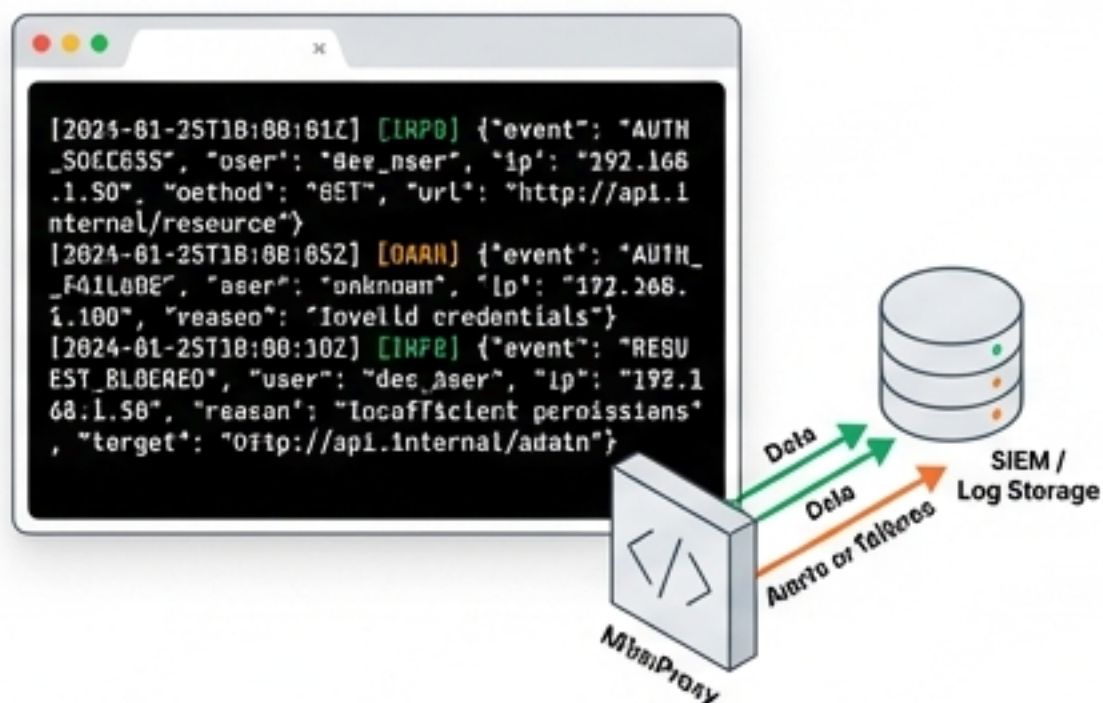
Role-Based Access Control (RBAC)

Enforces permissions per user via a hot-reloadable JSON configuration.



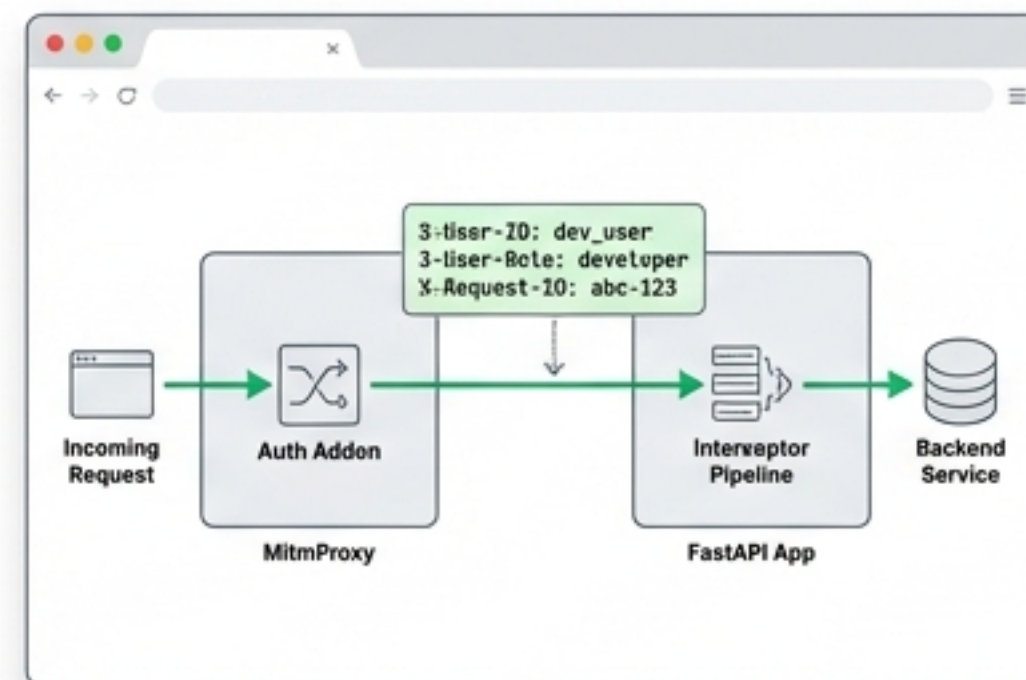
Audit Logging

Generates JSON-formatted trails for security reviews and SIEM ingestion.



Context Propagation

Seamless integration with the existing FastAPI interceptor pipeline via header injection.



The Current Risk Landscape: The Open Proxy

Problem	Severity	Impact
Unauthorized Access	Critical	Unrestricted access to internal resources.
No Accountability	High	Impossible to track "who did what".
Compliance Gaps	High	Lack of audit trails fails security reviews.
Resource Abuse	Medium	Unlimited usage without attribution.

Real-World Scenario

A developer shares the proxy URL for debugging. It gets shared further.

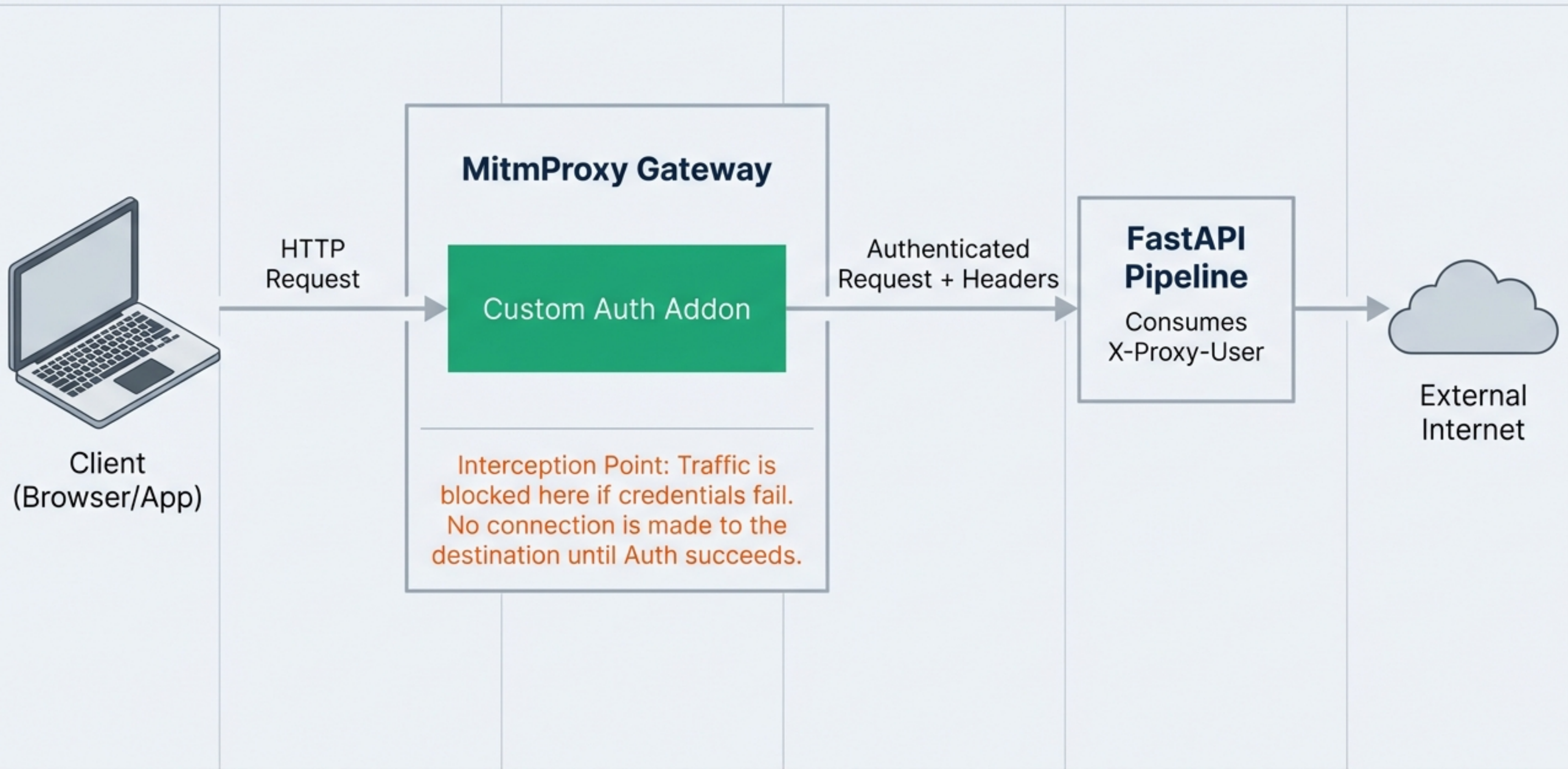
Unknown parties begin routing traffic, potentially accessing sensitive transformation features or using the server as an anonymous relay.

Strategic Solution Selection

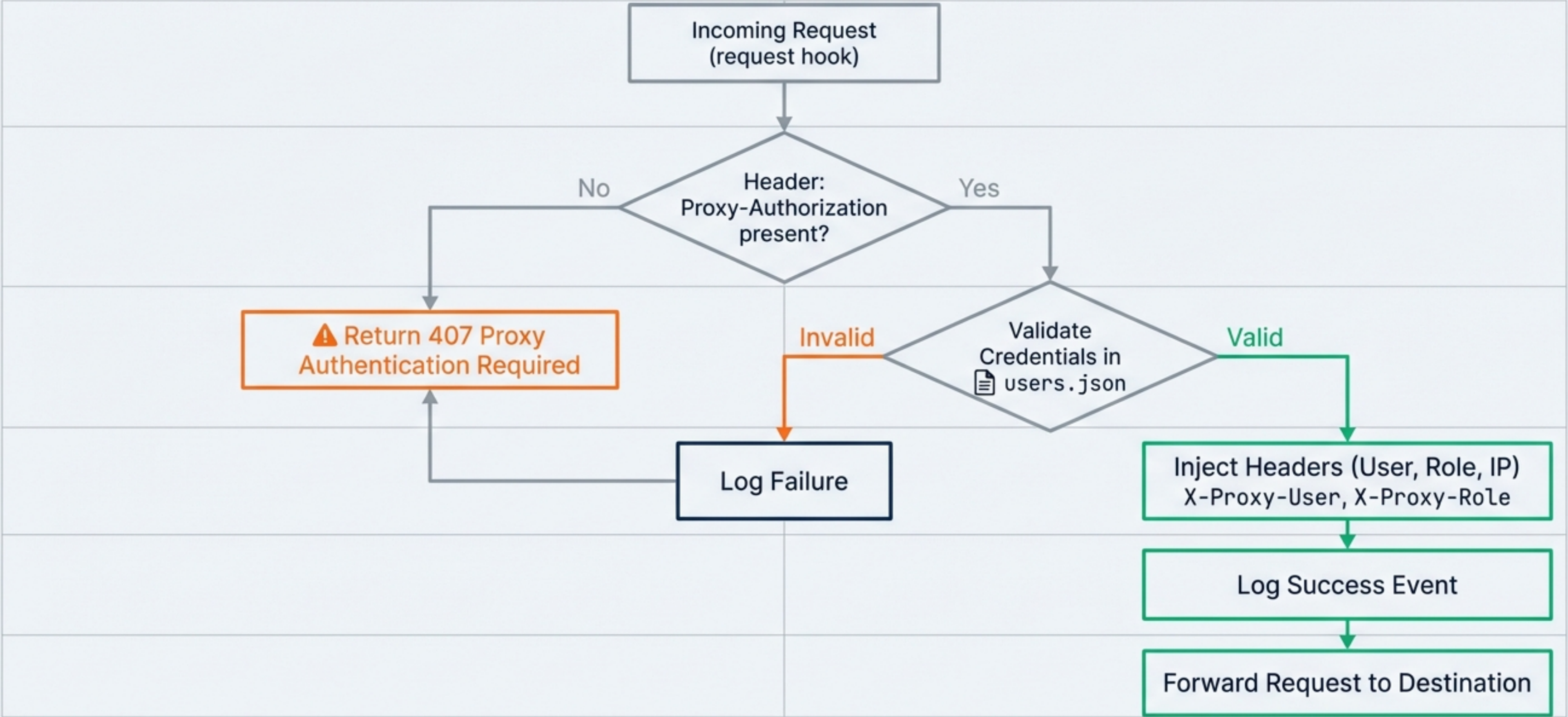
	Pros	Cons	Verdict
Built-in --proxyauth	Zero code, simple	Single user only, no logging	✗ Rejected
Nginx Reverse Proxy	Battle-tested	Adds infrastructure complexity, no internal integration	✗ Rejected
mTLS Certificates	Very secure	High management overhead (overkill)	✗ Rejected
Custom Python Addon	Native integration, Zero latency, Hot reload	Requires Python code	✓ Selected

Why Custom Wins: Native integration allows us to run inside the event loop with zero latency, while injecting `X-Proxy-User` headers for downstream apps.

System Architecture Context



The Authentication Logic Flow



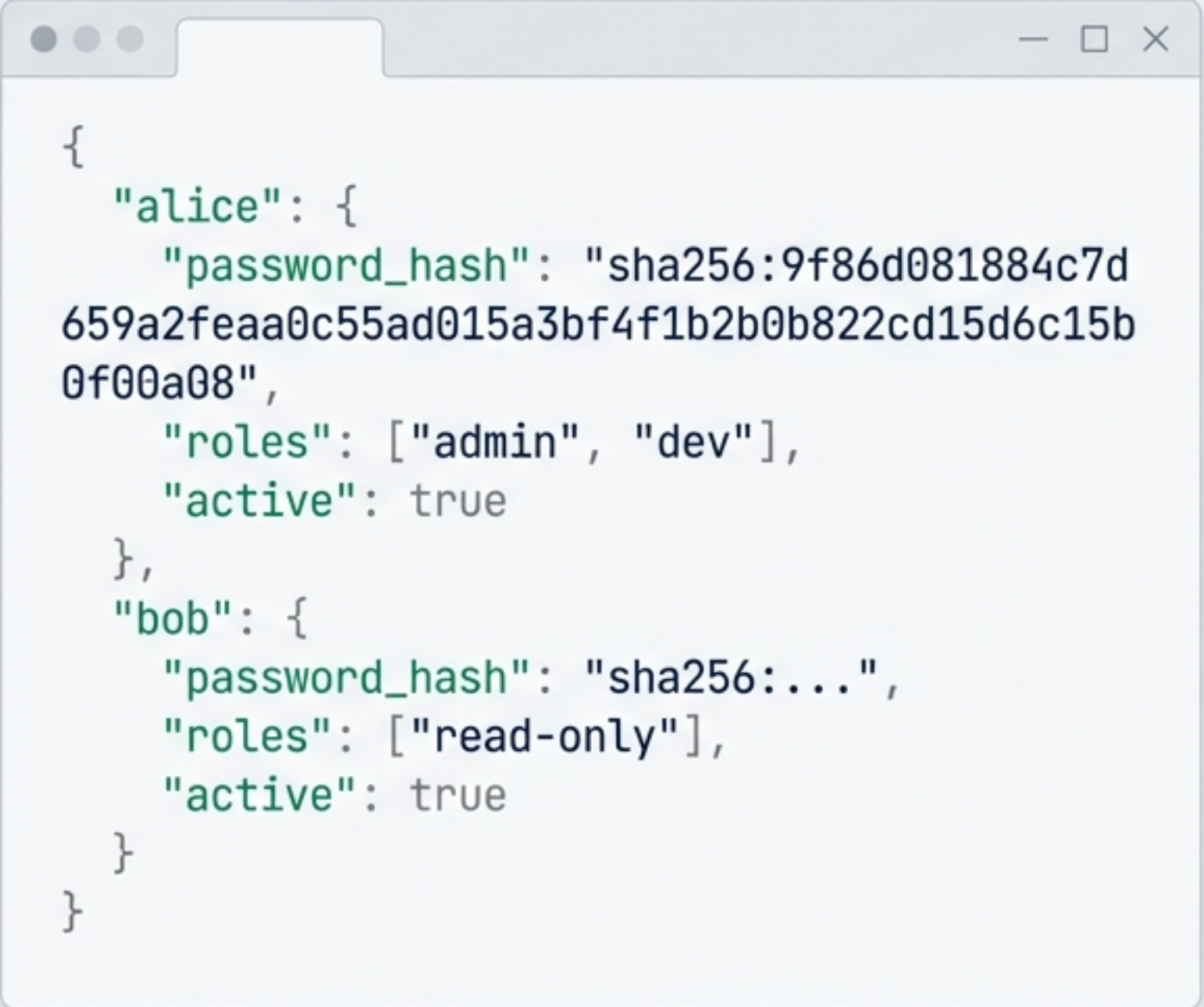
Data Structures: User Configuration

`/etc/mitmproxy/users.json`

The configuration drives the logic. Separating data from code allows for hot-reloading—the Python addon watches this file and updates permissions immediately without a service restart.

Key Fields:

- `password_hash`: SHA256 encoded.
 `password_hash`: SHA256 encoded.
- `roles`: Array of strings for granular permissions.
- `active`: Boolean to quickly disable access.



```
{
  "alice": {
    "password_hash": "sha256:9f86d081884c7d
659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b
0f00a08",
    "roles": ["admin", "dev"],
    "active": true
  },
  "bob": {
    "password_hash": "sha256:...",
    "roles": ["read-only"],
    "active": true
  }
}
```


Core Implementation: The Python Addon

File: /home/ubuntu/proxy_auth.py

```
def request(self, flow: http.HTTPFlow):  
    auth_header = flow.request.headers.get('Proxy-Authorization')  
  
    if not auth_header:  
        self.send_407(flow)  
        return  
  
    user = self.validate_creds(auth_header)  
    if not user:  
        logging.info(f'Auth failed from {flow.client_conn.address}')  
        self.send_407(flow)  
        return  
  
    # Success  
    flow.request.headers['X-Proxy-User'] = user.name
```

Event Hook:

Intercepts every HTTP flow.

Header Parsing:

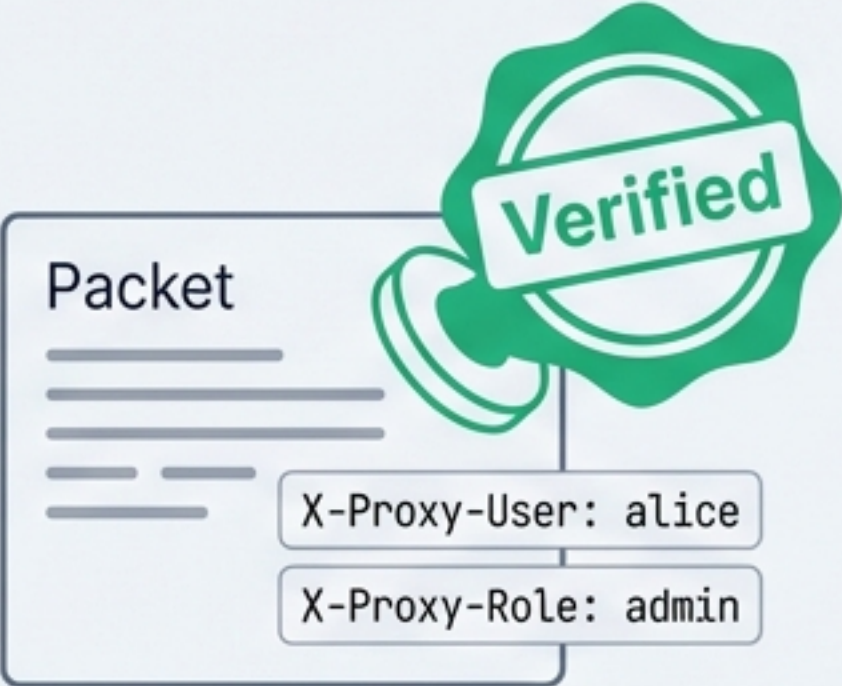
Extracts credentials.

Context Injection:

Enriches request before forwarding.

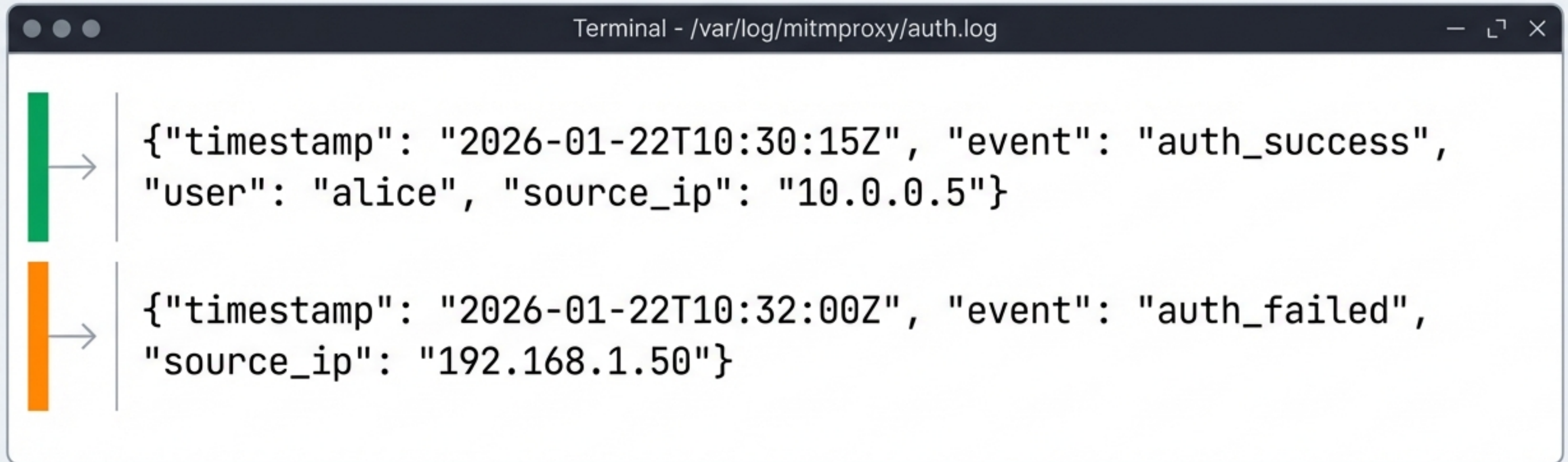
Context Propagation & Header Injection

Upon successful authentication, the addon injects specific headers. Downstream services trust these headers for their own authorization logic.

Header Name	Sample Value	Purpose	
X-Proxy-User	alice	Identity for audit trails.	
X-Proxy-Role	admin	Authorization decisions.	
X-Proxy-Source-IP	192.168.1.100	Forensics (Origin IP).	
X-Proxy-Auth-Time	2026-01-22T10:30:15Z	Session timing.	

Accountability: Audit Logging

File: /var/log/mitmproxy/auth.log

A terminal window titled "Terminal - /var/log/mitmproxy/auth.log" displays two lines of JSON Lines (JSONL) log data. The first line is highlighted with a green vertical bar and an arrow pointing to it, representing a successful authentication event. The second line is highlighted with an orange vertical bar and an arrow pointing to it, representing a failed authentication event. The JSON objects are: {"timestamp": "2026-01-22T10:30:15Z", "event": "auth_success", "user": "alice", "source_ip": "10.0.0.5"} and {"timestamp": "2026-01-22T10:32:00Z", "event": "auth_failed", "source_ip": "192.168.1.50"}.

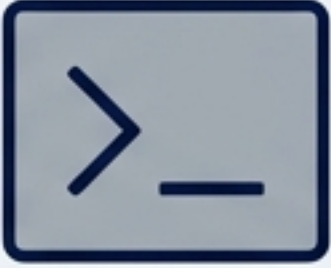
```
Terminal - /var/log/mitmproxy/auth.log

{"timestamp": "2026-01-22T10:30:15Z", "event": "auth_success",
"user": "alice", "source_ip": "10.0.0.5"}

{"timestamp": "2026-01-22T10:32:00Z", "event": "auth_failed",
"source_ip": "192.168.1.50"}
```

Format: JSON Lines (JSONL). Optimised for parsing by SIEM tools (Splunk, ELK). Captures both successes and failures to detect brute-force attacks.

Deployment & Operations

Service Management	User Management CLI	Log Maintenance
		
Service Management	User Management CLI	Log Maintenance
Systemd Unit: <code>mitmproxy.service</code> Ensures the service starts on boot and restarts automatically on failure.	Tool: <code>manage_users.py</code> Automates JSON manipulation and SHA256 hashing to prevent manual configuration errors.	Logrotate Config Prevents disk overflow. Rotates logs daily, keeping 14 days of history.

Security Considerations

Credential Storage

We strictly avoid Plain Text storage.



Policy: Minimum 8 characters enforced by CLI.

Transport Security (Crucial)

Basic Auth encodes credentials in Base64 (Not Encrypted). The connection TO the proxy must be secured.

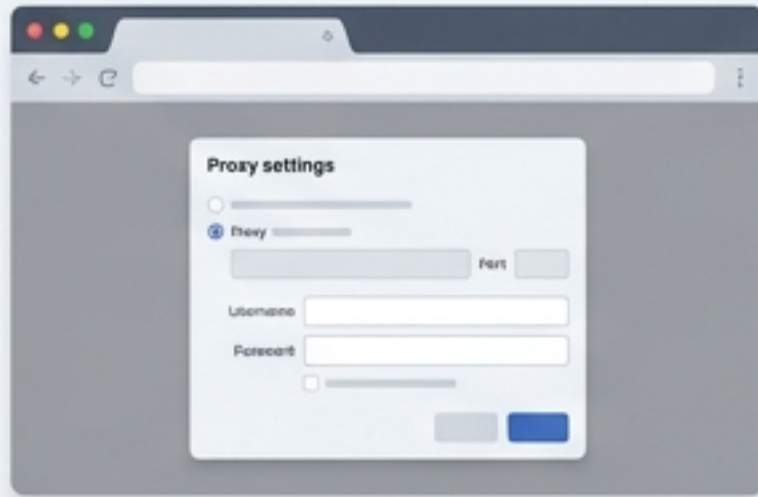


Required Mitigation

1. TLS Termination (HTTPS)
2. VPN / Private Network
3. Strict Firewall Rules

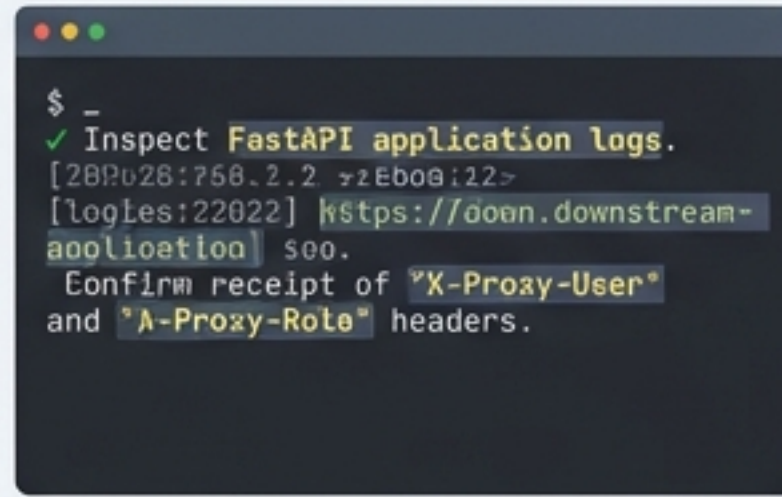
Testing & Verification

Manual Browser Check



- ✓ Configure browser proxy settings.
- ✓ Verify the system prompts for username/password.
- ✓ Confirm "407 Authorization Required" on invalid input.

Downstream Verification



- ✓ Inspect FastAPI application logs.
- ✓ Confirm receipt of "X-Proxy-User" and "X-Proxy-Role" headers.

Audit Trail Confirmation



- ✓ Trigger a login failure.
- ✓ Check "/var/log/mitmproxy/auth.log" to confirm the event was captured with the correct timestamp and IP.

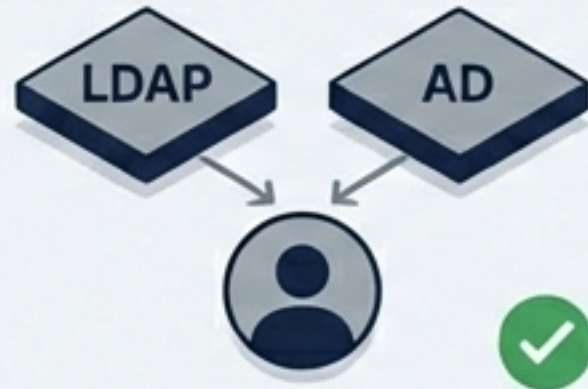
Future Roadmap & Enhancements

Medium Priority (Enterprise Scale)

Technical Grey

- **LDAP/AD Integration:**

For centralised Enterprise Single Sign-On.



- **Prometheus Metrics:**

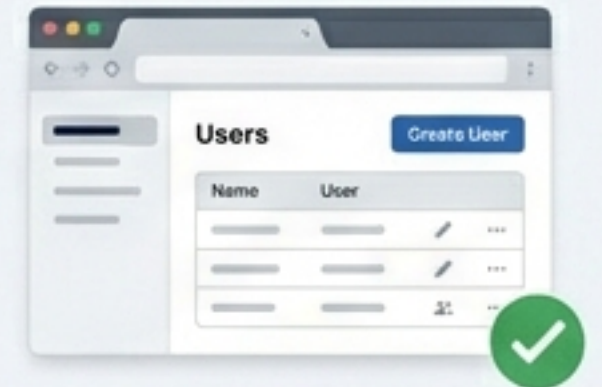
To track authentication success/failure rates and latency.



Low Priority (Quality of Life)

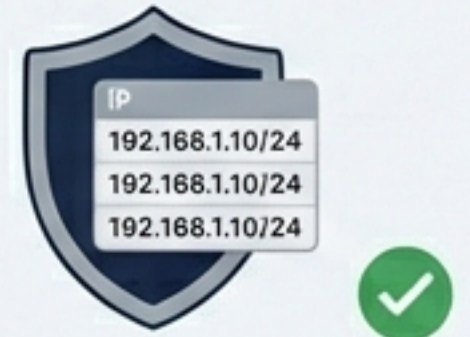
- **Web UI:**

A graphical interface to replace the CLI for user management.



- **IP Allowlists:**

Per-user IP restrictions.



- **Rate Limiting:**

Per-user throttling to prevent abuse.



Implementation Ready: Quick Reference

Client Connection Settings

Proxy Type: HTTP

Hostname: [Server-IP] or localhost

Port: 8080

Auth Type: Basic

Deliverables Summary

- ✓ Secure Python Addon
- ✓ JSON-based RBAC
- ✓ Full Audit Logging